# PostgreSQL Cheat Sheet

PostgreSQL 18 (examples run on 18.6) · AI-generated summary — verify against the PostgreSQL documentation · see the PostgreSQL pages for details

## PSQL — CONNECT

### Connect

```
psql -U postgres -d postgres -h host
psql "postgresql://user:pw@host:5432/db"
psql -f file.sql    psql -c "SELECT 1"
pg_isready -h 172.17.0.6 -p 5432
```

### URI parameters

sslmode (disable, prefer, require, verify-full), application_name, connect_timeout, target_session_attrs=read-write (primary from a host list). Password from PGPASSWORD or ~/.pgpass.

### Meta-commands

```
\conninfo  \l+  \c dbname  \dn  \dt shop.*
\d+ shop.products  \di \dv \ds \df  \du \dx
\x auto  \timing on  \watch 2  \sf func
\set VAR value  :VAR  :'VAR'  \gset
\copy tbl TO 'f.csv' CSV HEADER
```

## SQL — WRITE

### Identity and upsert

```
CREATE TABLE t_demo (
  id bigint GENERATED ALWAYS AS IDENTITY
     PRIMARY KEY,
  name text NOT NULL);
INSERT INTO t_demo (id, name)
  OVERRIDING SYSTEM VALUE VALUES (10, 'c')
  ON CONFLICT (id) DO UPDATE
  SET name = EXCLUDED.name RETURNING *;
```

### MERGE (15), RETURNING (17)

```
MERGE INTO t_demo t
  USING (VALUES (10,'cc')) v(id,name)
  ON t.id = v.id
  WHEN MATCHED THEN
    UPDATE SET name = v.name
  WHEN NOT MATCHED THEN INSERT ...
  RETURNING merge_action(), t.id;
```

### Note

RETURNING works on INSERT, UPDATE, DELETE and MERGE; OLD/NEW in RETURNING from 18. Also ON CONFLICT DO NOTHING, UPDATE ... FROM, DELETE ... USING.

## TYPES — CHOOSE

### Prefer

```
bigint GENERATED ALWAYS AS IDENTITY
numeric(p,s)    -- exact, for money
text            -- no length penalty
timestamptz     jsonb    uuid
int[]           int4range, tstzrange
```

### uuid and generated columns

```
gen_random_uuid()   uuidv7()  -- 18
gross numeric(10,2) GENERATED ALWAYS
  AS (round(net * (1 + vat_rate), 2))
  STORED
net_cents bigint GENERATED ALWAYS
  AS ((net * 100)::bigint) VIRTUAL
```

### Note

Identity replaces serial. uuidv7() is time-ordered; VIRTUAL columns (18, the default) are computed on read. Also inet, cidr, bytea; vector(n) via pgvector.

## INDEXES — CREATE

### Methods

```
btree   equality, ranges, sort (default)
hash    equality only
gin     arrays, jsonb, full-text
gist    geometry, ranges, nearest
spgist  tries, quadtrees
brin    huge, naturally ordered tables
```

### Variants

```
CREATE INDEX ON t_demo (lower(name));
CREATE INDEX ON t_demo (id)
  WHERE note IS NULL;
CREATE INDEX ON t_demo (name)
  INCLUDE (created);
CREATE INDEX CONCURRENTLY ...
```

### Note

CONCURRENTLY takes no write lock and cannot run inside a transaction. hnsw and ivfflat (pgvector) serve vector search.

## EXPLAIN — ANALYZE

### Options

```
EXPLAIN SELECT ...;  -- estimates only
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
EXPLAIN (ANALYZE, BUFFERS, VERBOSE,
  SETTINGS, WAL, SERIALIZE) SELECT ...;
EXPLAIN (FORMAT JSON) SELECT ...;
EXPLAIN (ANALYZE, COSTS OFF,
  TIMING OFF, SUMMARY OFF) SELECT ...;
```

### Note

ANALYZE executes the statement: wrap writes in BEGIN; ... ROLLBACK;. FORMAT also takes TEXT, XML, YAML.

### Plan nodes

Scans: Seq Scan, Index Scan, Index Only Scan, Bitmap Heap Scan, Parallel Seq Scan. Joins: Nested Loop, Hash Join, Merge Join.

## TRANSACTIONS — LOCK

### Isolation levels

READ COMMITTED (default): each statement sees data committed before it began. REPEATABLE READ: one snapshot per transaction. SERIALIZABLE: as if one at a time; retry on error 40001.

### Statements

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT * FROM shop.orders
  WHERE status = 'open' ORDER BY id
  FOR UPDATE SKIP LOCKED LIMIT 1;
SELECT pg_try_advisory_lock(42);
```

### Locks

Row: FOR UPDATE, FOR NO KEY UPDATE, FOR SHARE, FOR KEY SHARE, with NOWAIT or SKIP LOCKED. Table: ACCESS SHARE to ACCESS EXCLUSIVE. Blockers: pg_blocking_pids(pid), pg_locks.

## JSONB — QUERY

### Operators

```
-> ->>    field as jsonb / as text
#> #>>    path as jsonb / as text
@> <@     contains / is contained in
? ?| ?&   key / any key / all keys
|| - #-   merge / del key / del path
@? @@     jsonpath exists / predicate
```

### Functions

jsonb_set, jsonb_build_object, jsonb_agg, jsonb_each, jsonb_array_elements, jsonb_path_query. SQL/JSON: JSON_VALUE, JSON_QUERY, JSON_EXISTS, JSON_TABLE (17).

### Index

```
USING gin (col jsonb_path_ops)
CREATE INDEX ON t ((col->>'key'));
```

## MONITORING — OBSERVE

### Activity

```
SELECT pid, state, wait_event,
  pg_blocking_pids(pid)
  FROM pg_stat_activity
  WHERE state <> 'idle';
SELECT pg_cancel_backend(pid);
SELECT pg_terminate_backend(pid);
```

### Tables

```
SELECT relname, n_dead_tup, last_autovacuum
  FROM pg_stat_user_tables;
VACUUM (VERBOSE, ANALYZE) shop.orders;
```

### Settings

```
SELECT name, setting, pending_restart
  FROM pg_settings
  WHERE name = 'work_mem';
ALTER SYSTEM SET work_mem = '32MB';
SELECT pg_reload_conf();
```

## BACKUP — REPLICATE

### Dump and restore

```
pg_dump -U postgres -Fc -n shop \
  -f shop.dump postgres
pg_dumpall -U postgres --globals-only
pg_restore -U postgres -d newdb -j 4 \
  --no-owner shop.dump
```

### Base backup / standby

```
pg_basebackup -U postgres -D /backup/base
  -Fp -X stream -c fast -P
pg_basebackup -h primary -U replicator
  -D datadir -R -X stream -C -S standby1
```

### Replication checks

```
SELECT application_name, state,
  sync_state, replay_lag
  FROM pg_stat_replication;  -- primary
SELECT pg_is_in_recovery();
SELECT pg_promote();
```

### Note

Logical replication (CREATE PUBLICATION / SUBSCRIPTION) needs wal_level = logical.